BÁO CÁO BÀI THỰC HÀNH LAB 02

MongoDB Atlas Free với PyMongo trên Google Cloud Shell

Môn: Introduction to Data Science — Học viện Công nghệ Bưu chính Viễn thông
Sinh viên: Nguyễn Danh Thái MSSV: B23DCKH105

---

1. Thông tin chung

Bài thực hành dựng cơ sở dữ liệu NoSQL trên MongoDB Atlas Free gồm ba collection customers, products, orders — mỗi đơn hàng chứa mảng items nhúng — rồi thực hiện CRUD, truy vấn trường lồng nhau, aggregation, $lookup và index. Toàn bộ 22 yêu cầu N01–N22 đã cài đặt và chạy kiểm chứng.

- Dịch vụ: MongoDB Atlas M0 Free, database sales_nosql; kết nối bằng PyMongo từ Google Cloud Shell, URI nhập qua getpass nên không lưu mật khẩu trong mã nguồn.
- Phần bắt buộc N01–N15: n_answers.py trên bộ dữ liệu mẫu 5 khách / 5 sản phẩm / 6 đơn. Phần bổ sung N16–N22: n_homework.py trên dữ liệu mở rộng do generate_data.py sinh.
- Kịch bản hỗ trợ: connect.py (ping + trả về database), seed.py (nạp dữ liệu mẫu), lab_queries.py (ví dụ trong đề), generate_data.py (dữ liệu mở rộng).

2. Tiêu chí 1 — Thiết lập và seed dữ liệu (2,0 điểm)

connect.py dùng getpass nhận URI, tạo MongoClient với tham số timeout 12.000 ms rồi gọi client.admin.command('ping') để chắc chắn cluster trả lời trước khi dùng. Mọi lỗi thường gặp được phân biệt rõ: timeout là do IP chưa allowlist hoặc DNS, còn AuthenticationFailed là do user hoặc mật khẩu (ký tự đặc biệt phải URL-encode).

```
from getpass import getpass
from pymongo import MongoClient

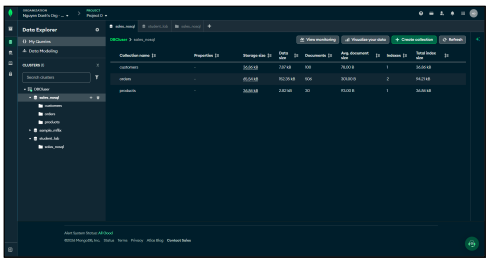
def get_database():
    uri = getpass('Paste complete Atlas URI: ')
    client = MongoClient(uri, serverSelectionTimeoutMS=12000)
    client.admin.command('ping')
    print('Connected successfully')
    return client['sales_nosql']
```

seed.py nạp đúng bộ dữ liệu của đề và dừng nếu database đã có dữ liệu, nên không bao giờ trộn dữ liệu mẫu với dữ liệu mở rộng. Mỗi order lưu items là mảng document nhúng gồm product_id, name, price, quantity — giá và tên được sao lại tại thời điểm mua.

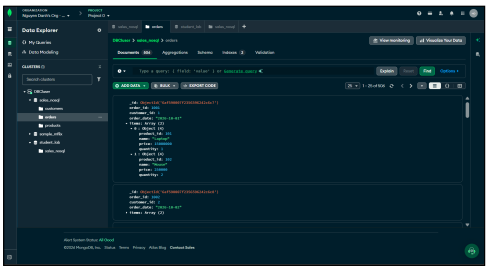
Bảng 1. Số document sau khi chạy seed.py

Collection	Kỳ vọng của đề	Thực tế	Kết quả
customers	5	5	khớp
products	5	5	khớp
orders	6	6	khớp

Kiểm tra trên Atlas → Data Explorer: mở một order thấy trường items là array các document con, không có bảng trung gian như order_items của mô hình quan hệ.



Hình 1. Cluster DSCluster (M0 Free) và database sales_nosql



Hình 2. Data Explorer: orders có mảng items nhúng

```
(.venv) nguyendanhthaib23dckh105@cloudshell:~/data_science/nosql_lab/KHDL_PTIT_Lab/lab02_nosql$ python3 connect.py
Paste complete Atlas URI:

Database: sales_nosql
Collections: (rỗng)
Số document: {'customers': 0, 'products': 0, 'orders': 0}

(.venv) nguyendanhthaib23dckh105@cloudshell:~/data_science/nosql_lab/KHDL_PTIT_Lab/lab02_nosql$ python3 seed.py
Paste complete Atlas URI:

Connected successfully
customers 5
products 5
orders 6
Mẫu document orders (items nhúng):
   {'product_id': 101, 'name': 'Laptop', 'price': 15000000, 'quantity': 1}
   {'product_id': 102, 'name': 'Mouse', 'price': 250000, 'quantity': 2}
```

Hình 3. connect.py (kết nối, 3 collection rỗng) và seed.py (5/5/6, mẫu items nhúng)

3. Tiêu chí 2 — CRUD và truy vấn lọc (2,0 điểm)

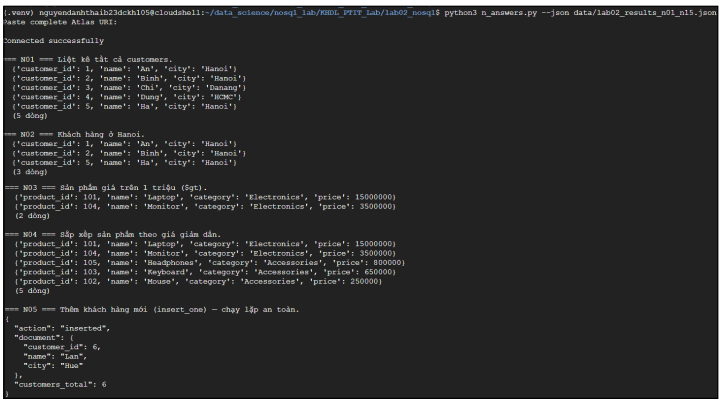
Nhóm N01–N08 gồm bốn truy vấn đọc và ba thao tác ghi. Các thao tác ghi được viết để chạy lặp an toàn trên cùng một database.

Bảng 2. Các câu N01–N08 và kết quả thực tế

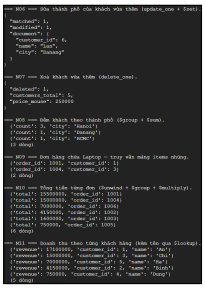
Mã	Nội dung	Cách thực hiện	Kết quả
N01	Liệt kê tất cả customers	find({})	5 document
N02	Khách hàng ở Hanoi	find({city: Hanoi})	3 document
N03	Sản phẩm giá trên 1 triệu	{price: {$gt: 1000000}}	2 document
N04	Sắp xếp theo giá giảm dần	sort(price, -1)	5 document
N05	Thêm khách hàng mới	insert_one	1 document
N06	Sửa thành phố	update_one + $set	matched 1
N07	Xoá khách vừa thêm	delete_one	deleted 1
N08	Đếm khách theo thành phố	$group + $sum	3 nhóm

```
pipeline = [
    {'$group': {'_id': '$city', 'count': {'$sum': 1}}},
    {'$sort': {'count': -1, '_id': 1}},
    {'$project': {'_id': 0, 'city': '_id', 'count': 1}},
]
```

- N08 cho kết quả: Hanoi 3, Danang 1, HCMC 1; tổng bằng đúng số khách hàng trong bộ dữ liệu mẫu.
- N05 chỉ insert khi customer_id = 6 chưa tồn tại, N06 $set city = Danang, N07 delete_one đúng khách đó nên chạy lại nhiều lần vẫn về 5 khách hàng.
- Truy vấn mảng nhúng dùng đường dẫn items.product_id nên không cần join: N09 tìm được đơn chứa Laptop bằng một lần đọc collection orders.



Hình 4. n_answers.py: kết quả N01–N05 trên terminal Cloud Shell



Hình 5. N05–N11: CRUD, đếm theo thành phố, mảng items



Hình 6. N13 lọc sau $group, N14 tạo index

4. Tiêu chí 3 — Aggregation và $lookup (3,0 điểm)

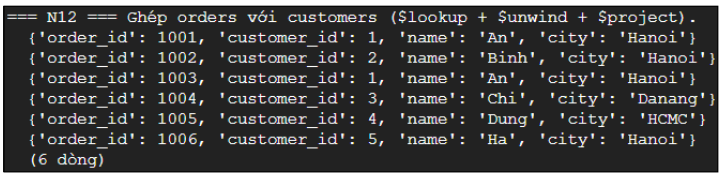
N10 dùng $unwind để tách mảng items rồi $group lại theo order_id, nhân price với quantity bằng $multiply. N11 gom theo customer_id và $lookup sang customers để lấy tên; N12 ghép orders với customers; N13 lọc nhóm bằng $match sau $group.

```
pipeline = [
    {'$unwind': '$items'},
    {'$group': {'_id': '$order_id',
                'total': {'$sum': {'$multiply': [
                    '$items.price', '$items.quantity']}}}},
    {'$sort': {'total': -1}},
]
```

Bảng 3. Kết quả N10 — tổng tiền từng đơn

Mã đơn	Tổng tiền (đồng)
1001	15.500.000
1004	15.000.000
1006	7.000.000
1002	4.150.000
1003	1.600.000
1005	750.000
Tổng	44.000.000

- Kiểm tra chéo đơn 1001: 1 Laptop 15.000.000 + 2 Mouse 250.000 = 15.500.000, khớp kết quả máy; tổng cả 6 đơn là 44.000.000 đồng.
- N11 — doanh thu theo khách: An 17.100.000, Chi 15.000.000, Ha 7.000.000, Binh 4.150.000, Dung 750.000.
- N12 $lookup + $unwind + $project trả về 6 dòng dạng {order_id, name, city}, tương đương INNER JOIN của SQL.
- N13 — khách có từ 2 đơn: An (2); các khách còn lại chỉ có 1 đơn nên bị $match loại.



Hình 7. N12 — $lookup + $unwind, 6 dòng ghép

5. Tiêu chí 4 — Index và mô hình dữ liệu (1,5 điểm)

N14 tạo index trên orders.customer_id đúng như đề. N21 đọc explain() để so sánh kế hoạch khi truy vấn theo trường có index và trường không có index.

```
db.orders.create_index([('customer_id', 1)], name='idx_orders_customer')
plan = db.orders.find({'customer_id': 7}).explain()
print(plan['queryPlanner']['winningPlan'])   # FETCH <- IXSCAN
```

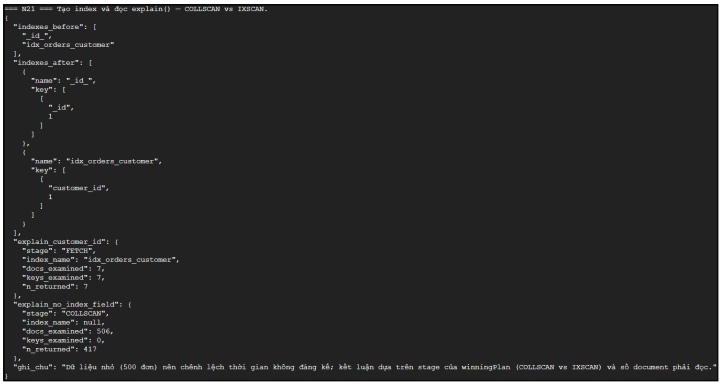
Bảng 4. Kế hoạch thực thi khi có index và khi không có index

Truy vấn	Stage	Document đọc	Diễn giải
customer_id (có index)	FETCH	7	IXSCAN trên index rồi FETCH
order_date (không index)	COLLSCAN	506	quét toàn collection rồi lọc

- Với 506 đơn, chênh lệch thời gian chạy dưới ngưỡng đo nên kết luận dựa trên winningPlan và số document phải đọc: truy vấn có index chỉ đọc 7 document, truy vấn không index quét 506 và trả về 417.
- N15 — embedded (items nằm trong order) đọc cả đơn bằng một truy vấn, ghi nguyên tử, nhưng lặp name/price và bị giới hạn 16 MB mỗi document; reference (chỉ lưu product_id) không lặp dữ liệu nhưng phải $lookup mỗi lần đọc.
- N20 — đổi price trong products từ 15.000.000 lên 16.000.000 đồng thì items.price trong đơn cũ vẫn giữ nguyên 15.000.000 đồng: dữ liệu nhúng là bản sao tại thời điểm mua, đúng cho báo cáo doanh thu lịch sử nhưng phải chấp nhận không đồng bộ với bảng giá hiện hành.



Hình 8. N15 mô hình nhúng/tham chiếu và N20 đổi giá trong products



Hình 9. explain(): đọc 7 document khi có index, 506 khi COLLSCAN

6. Tiêu chí 5 — Bài mở rộng và báo cáo (1,5 điểm)

generate_data.py sinh dữ liệu mở rộng với random.seed(2026) nên tái lập được: 100 khách, 30 sản phẩm, 500 đơn, mỗi đơn 1–5 sản phẩm (1.521 dòng hàng), 8 khách không có đơn và 4 sản phẩm không bán được.

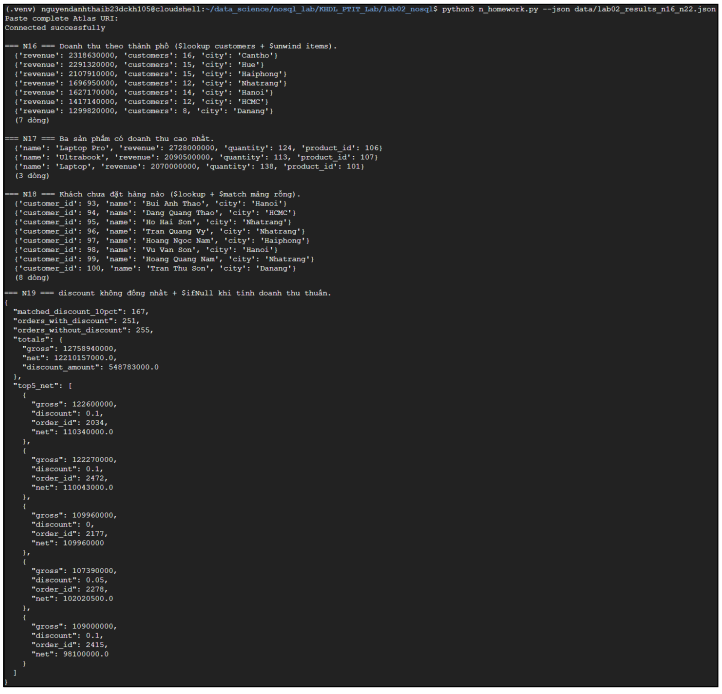
Bảng 5. Quy mô dữ liệu mở rộng so với yêu cầu của đề

Đối tượng	Yêu cầu tối thiểu	Thực tế
Khách hàng	100	100
Sản phẩm	30	30
Đơn hàng	500	500
Dòng items	1.000	1.521

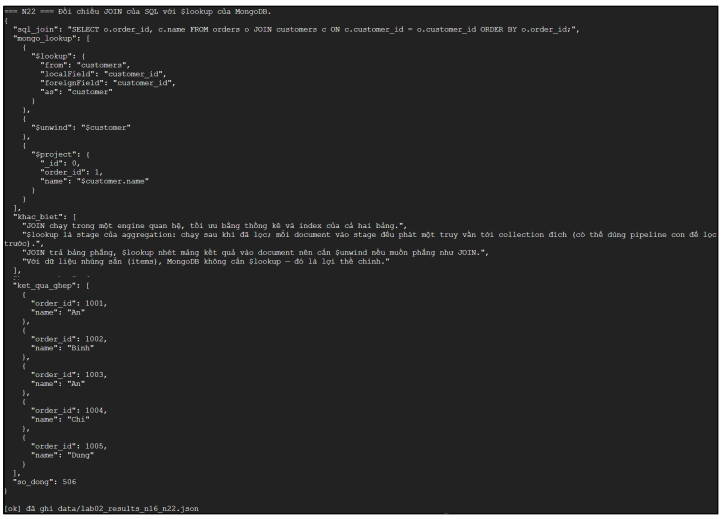
- N16 — doanh thu theo thành phố dẫn đầu: Cantho 2.318.630.000, Hue 2.291.320.000 (7 thành phố, tổng khớp doanh thu toàn hệ thống).
- N17 — ba sản phẩm doanh thu cao nhất: Laptop Pro 2.728.000.000; Ultrabook 2.090.500.000; Laptop 2.070.000.000.
- N18 — $lookup orders rồi $match mảng rỗng tìm được 8 khách chưa đặt hàng, đúng bằng số khách cố ý để trống khi sinh dữ liệu.
- N19 — discount không đồng nhất: 251 đơn có discount, 255 đơn không có trường này; $ifNull quy trường thiếu về 0 nên doanh thu gộp 12.758.940.000 đồng, doanh thu thuần 12.210.157.000 đồng (giảm 548.783.000 đồng).
- N22 — JOIN của SQL và $lookup cùng trả 506 dòng ghép, nhưng $lookup là stage chạy trong pipeline nên lọc trước sẽ rẻ hơn; dữ liệu nhúng sẵn như items không cần $lookup, đó là khác biệt cốt lõi so với mô hình quan hệ.



Hình 10. generate_data.py: 100 khách / 30 sản phẩm / 500 đơn



Hình 11. N16–N19 trên dữ liệu mở rộng



Hình 12. N22 — đối chiếu JOIN của SQL với $lookup

7. Danh mục ảnh minh chứng

Ảnh chụp thật trên Cloud Shell và Atlas, đã chèn vào mục 2–6.

Bảng 6. Các ảnh minh chứng kèm theo bài nộp

Tệp trong evidence/	Nội dung ảnh (tóm tắt)	Mục
01_cluster.png	Cluster DSCluster (M0 Free) và database sales_nosql	2
02_data_explorer.png	Data Explorer: document orders có mảng items nhúng	2
03_connect_seed.png	connect.py và seed.py: kết nối, nạp bộ dữ liệu mẫu 5/5/6	2
04_crud_n01_n05.png	n_answers.py: kết quả N01–N05	3
05_crud_n05_n11.png	n_answers.py: kết quả N05–N11	3
07_lookup_n12.png	N12 — $lookup + $unwind + $project, 6 dòng ghép	4
08_n13_n14_index.png	N13 lọc sau $group và N14 tạo index orders.customer_id	3
09_n15_n20_mo_hinh.png	N15 mô hình nhúng/tham chiếu và N20 đổi giá trong products	5
10_explain_n21.png	explain(): 7 document khi có index, 506 khi COLLSCAN	5
11_expanded_counts.png	generate_data.py: sinh 100 khách / 30 sản phẩm / 500 đơn	6
12_n16_n19.png	N16–N19 trên dữ liệu mở rộng	6
13_n22_lookup.png	N22 — đối chiếu JOIN của SQL với $lookup	6

8. Kết luận

Đã hoàn thành đủ 22 yêu cầu: cluster Atlas Free và kết nối an toàn bằng getpass, seed 5/5/6 với items nhúng, CRUD và truy vấn mảng, aggregation có $unwind/$group/$lookup, index kèm explain, mở rộng 500 đơn với discount không đồng nhất. Bài học: chọn nhúng hay tham chiếu là quyết định về mô hình dữ liệu, và đánh giá index trên dữ liệu nhỏ phải dựa vào stage của explain chứ không phải thời gian chạy.